

-- 1. REGIONS

```
CREATE TABLE regions (  
    code VARCHAR PRIMARY KEY,  
    nom VARCHAR  
);
```

-- 2. DEPARTEMENTS

```
CREATE TABLE departements (  
    code VARCHAR PRIMARY KEY,  
    region VARCHAR REFERENCES regions(code),  
    nom VARCHAR  
);
```

-- 3. COMMUNES

```
CREATE TABLE communes (  
    code INTEGER PRIMARY KEY,  
    departement VARCHAR REFERENCES departements(code),  
    nom VARCHAR  
);
```

-- 4. SITES

```
CREATE TABLE sites (  
    idSite INTEGER PRIMARY KEY,  
    nom VARCHAR,  
    codeCommune INTEGER REFERENCES communes(code),  
    dateDeclaration DATE,  
    typeEau VARCHAR,  
    longitude FLOAT, -- Je garde NUMERIC pour les virgules, INTEGER arrondirait trop  
    latitude FLOAT  
);
```

-- 5. EVENEMENTS

```
CREATE TABLE evenements (  
    idEvenement INTEGER PRIMARY KEY,  
    idSite INTEGER REFERENCES sites(idSite),  
    evenement VARCHAR,  
    debut DATE,  
    fin DATE,  
    mesure VARCHAR  
);
```

-- 6. ANALYSES

```
CREATE TABLE analyses (  
    idAnalyse INTEGER PRIMARY KEY,  
    idSite INTEGER REFERENCES sites(idSite),  
    datePrelevement DATE,
```

```

    enterocoques INTEGER,
    escherichia INTEGER
);
CREATE TEMPORARY TABLE temp_sites (
    saison_text TEXT,
    region_text TEXT,
    dept_text TEXT,
    code_unique TEXT,
    prec_code TEXT,
    evolution TEXT,
    nom_site TEXT,
    code_insee TEXT,  -- C'est le lien vers la commune
    nom_commune TEXT,
    date_decl TEXT,
    type_eau TEXT,
    longitude TEXT,
    latitude TEXT
);
SELECT * FROM temp_analyses
copy temp_sites FROM
'C:/Users/Public/liste-des-sites-de-baignade-saison-balneaire-2024.csv' WITH (FORMAT
csv, HEADER true, DELIMITER ';', ENCODING 'WINDOWS-1252');
INSERT INTO sites (nom, codeCommune, dateDeclaration, typeEau, longitude, latitude)
SELECT
    nom_site,
    CAST(code_insee AS INTEGER),      -- Convertit le texte "1010" en nombre
    TO_DATE(date_decl, 'DD/MM/YYYY'), -- Convertit "01/07/2019" en date
    type_eau,
    REPLACE(longitude, ',', '.'):DECIMAL, -- Change la virgule en point pour le GPS
    REPLACE(latitude, ',', '.'):DECIMAL
FROM temp_sites;
-- Création du sac temporaire
CREATE TEMPORARY TABLE temp_geo (
    code_dept TEXT,
    nom_dept TEXT,
    code_region TEXT,
    nom_region TEXT
);
-- Chargement du fichier (Attention au nom du fichier, vérifie-le !)
COPY temp_geo
FROM 'C:/Users/Public/departements-france.csv'
WITH (FORMAT csv, HEADER true, DELIMITER ';', ENCODING 'UTF8');
-- Création du sac temporaire
CREATE TEMPORARY TABLE temp_evenements (
    saison TEXT,
    region TEXT,
    departement TEXT,

```

```

code_site TEXT,    -- C'est le code "FRK..." important pour le lien
type_event TEXT,
date_debut TEXT,
date_fin TEXT,
mesure TEXT
);

-- Chargement
COPY temp_evenements
FROM 'C:/Users/Public/saison-balneaire-2024-informations-sur-la-saison.csv'
WITH (FORMAT csv, HEADER true, DELIMITER ';', ENCODING 'WINDOWS-1252');
DROP TABLE IF EXISTS temp_analyses;

CREATE TEMPORARY TABLE temp_analyses (
    saison TEXT,
    region TEXT,
    departement TEXT,
    code_site TEXT,
    date_prelevement TEXT,
    resultat_entero TEXT,
    resultat_esch TEXT,
    statut TEXT,
    -- On met EXACTEMENT 3 poubelles car on voit 3 point-virgules à la fin
    poubelle1 TEXT,
    poubelle2 TEXT,
    poubelle3 TEXT
);

COPY temp_analyses
FROM 'C:/Users/Public/saison-balneaire-2024-resultats-danalyses.csv'
WITH (FORMAT csv, HEADER true, DELIMITER ';', ENCODING 'WINDOWS-1252');
INSERT INTO regions (code, nom)
SELECT DISTINCT
    code_region,
    nom_region
FROM temp_geo;
INSERT INTO departements (code, region, nom)
SELECT
    code_dept,
    code_region,
    nom_dept
FROM temp_geo;
INSERT INTO communes (code, departement, nom)
SELECT DISTINCT
    CAST(code_insee AS INTEGER),
    -- La correction magique : si c'est "042", on garde "42"
    CASE
        WHEN LENGTH(dept_text) = 3 AND dept_text LIKE '0%' THEN RIGHT(dept_text, 2)

```

```

        ELSE dept_text
    END,
    nom_commune
FROM temp_sites
WHERE code_insee ~ '^[0-9]+$'; -- Toujours le filtre anti-Corse
INSERT INTO sites (idSite, nom, codeCommune, dateDeclaration, typeEau, longitude,
latitude)
SELECT
    ROW_NUMBER() OVER (ORDER BY nom_site) AS idSite, -- On génère nous-même les
numéros 1, 2, 3...
    nom_site,
    CAST(code_insee AS INTEGER),
    TO_DATE(date_decl, 'DD/MM/YYYY'),
    type_eau,
    REPLACE(longitude, ',', '. '::DECIMAL,
    REPLACE(latitude, ',', '. '::DECIMAL
FROM temp_sites
WHERE code_insee ~ '^[0-9]+$'; -- Toujours le filtre pour virer la Corse
-- Vérif Régions
SELECT * FROM regions ;

-- Vérif Départements
SELECT * FROM departements ;

-- Vérif Communes
SELECT * FROM communes ;

-- Vérif Sites (Le plus important !)
SELECT * FROM sites ;

-- Vérif Analyses et Evénements (Peut-être vide si tu n'as pas encore fait l'étape 5 et 6)
SELECT * FROM analyses ;
SELECT * FROM evenements ;
INSERT INTO evenements (idEvenement, idSite, evenement, debut, fin, mesure)
SELECT
    ROW_NUMBER() OVER (ORDER BY t_evt.date_debut) AS idEvenement, -- Le
générateur de numéros (1, 2, 3...)
    s.idSite,
    t_evt.type_event,
    TO_DATE(t_evt.date_debut, 'DD/MM/YYYY'),
    TO_DATE(t_evt.date_fin, 'DD/MM/YYYY'),
    t_evt.mesure
FROM temp_evenements t_evt
JOIN temp_sites t_site ON t_evt.code_site = t_site.code_unique
JOIN sites s ON t_site.nom_site = s.nom;
INSERT INTO analyses (idAnalyse, idSite, datePrelevement, enterocoques, escherichia)
SELECT

```

```

    ROW_NUMBER() OVER (ORDER BY t_ana.date_prelevement) AS idAnalyse, -- Le
générateur de numéros
    s.idSite,
    TO_DATE(t_ana.date_prelevement, 'DD/MM/YYYY'),
    CAST(t_ana.resultat_entero AS INTEGER),
    CAST(t_ana.resultat_esch AS INTEGER)
FROM temp_analyses t_ana
JOIN temp_sites t_site ON t_ana.code_site = t_site.code_unique
JOIN sites s ON t_site.nom_site = s.nom
WHERE t_ana.resultat_entero ~ '^[0-9]+$'
    AND t_ana.resultat_esch ~ '^[0-9]+$';
DROP TABLE IF EXISTS temp_analyses;
DROP TABLE IF EXISTS temp_evenements;
DROP TABLE IF EXISTS temp_sites;
DROP TABLE IF EXISTS temp_geo;
SELECT 'Regions' as Nom_Table, COUNT(*) as Nombre_Lignes FROM regions
UNION ALL
SELECT 'Departements', COUNT(*) FROM departements
UNION ALL
SELECT 'Communes', COUNT(*) FROM communes
UNION ALL
SELECT 'Sites', COUNT(*) FROM sites
UNION ALL
SELECT 'Evenements', COUNT(*) FROM evenements
UNION ALL
SELECT 'Analyses', COUNT(*) FROM analyses;
SELECT
    s.nom as Nom_Plage,
    c.nom as Ville,
    a.datePrelevement,
    a.escherichia
FROM sites s
JOIN communes c ON s.codeCommune = c.code
JOIN analyses a ON s.idSite = a.idSite
LIMIT 10;

```